

Relatório de Auditoria de Segurança — MaisLouvor (+Louvor)

Data: 29/08/2026

Commit auditado: e4f8cc4be498

ESCOPO AUDITADO

Frontend React/Vite, cliente Supabase/PostgREST, 21 migrations SQL, 11 Edge Functions Deno, worker FastAPI/Docker, três buckets Storage, configuração de deploy, documentação, árvore e histórico Git e bundle de produção.

Nota metodológica

A auditoria mapeou as cinco categorias à stack detectada: isolamento por RLS e namespaces de Storage; gates React cruzados com policies/Edge Functions; IDOR em cada handler e relação por ID; segredos no tree, histórico e bundle; e XSS em sinks HTML/URL e respostas. Somente condições demonstráveis no código foram classificadas como achado.

CLASSIFICAÇÃO: USO INTERNO • recomenda-se tratar P1 antes de nova publicação

Resumo executivo

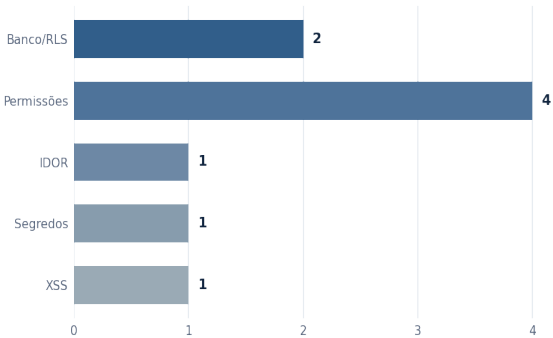
Foram confirmados 9 achados: 1 crítico, 5 altos e 3 médios. O risco dominante é uma cadeia de tomada de conta que transforma administração de um time em controle da identidade Supabase global. Também há quebra de isolamento em Storage e relações, endpoints de IA públicos, execução de javascript: armazenado e divergências entre gates do navegador e validações server-side.



Distribuição por severidade



Achados por categoria



Riscos centrais

Prioridade	Risco	Efeito
P1	Identidade global administrável por admin de time	Tomada integral de conta
P1	RLS incompleta em Storage e relações	Vazamento/adulteração cross-tenant
P1	URL armazenada executável	Roubo de sessão após clique
P1	IA pública sem quota	Custo e indisponibilidade



Stack detectada e mapeamento metodológico

Camada	Tecnologia detectada	Evidência
Linguagens	TypeScript/TSX, SQL/PLpgSQL, Python 3.11	package.json; migrations; demucs-worker/Dockerfile
Frontend	React 18, Vite 5, React Router 6, TanStack Query, Tailwind/shadcn, PWA	package.json:44-67; vite.config.ts
Backend	Supabase Edge Functions em Deno + FastAPI worker	supabase/functions/*; demucs-worker/app.py
Banco/query builder	PostgreSQL/Supabase; supabase-js/PostgREST; sem ORM tradicional	@supabase/supabase-js; chamadas .from/.select/.update
Auth	Supabase Auth/JWT; auth.uid() em RLS; getClaims em Edge; Bearer estático no worker	client.ts; migrations; handlers; app.py
Isolamento	RLS por team_id, membership/profile_id e namespace de Storage	is_team_member, can_edit_team, has_team_role
Deploy	Supabase config + Dockerfile do worker + publicação Lovable	supabase/config.toml; demucs-worker/Dockerfile; README
Ausentes	Sem CI versionada, Helm, Terraform, docker-compose ou charts	Inventário completo do tree

Como cada categoria foi aplicada

Categoria solicitada	Equivalente nesta stack
1. Banco sem tranca	RLS PostgreSQL em tabelas + policies de storage.objects + filtros team_id/profile_id
2. Permissão no navegador	isAdmin/canEdit/readOnly no React versus RLS, getClaims e roles nas Edge Functions
3. IDOR	IDs em rotas React/PostgREST, bodies de Edge Functions, relações FK e job IDs FastAPI
4. Chaves expostas	.env, Deno.env, Docker env, docs/config, dois commits e artefatos dist
5. XSS	Sinks React/DOM, URLs href/src/window.open, chart CSS, texto/HTML de funções

Limitações e estado de verificação

- Auditoria estática do commit informado; não foram executados testes destrutivos contra o projeto Supabase ao vivo.
- O estado implantado de policies, grants, Auth e secrets não foi consultado; as migrations versionadas foram tratadas como fonte de verdade.
- Cinco migrations estão vazias/quase vazias e types.ts contém colunas sem migration correspondente (por exemplo, notes_author_id, sender_profile_id, lyrics_text e scroll_speed), indicando drift de schema.
- supabase/config.toml cita transcribe-sync, mas não há handler correspondente no repositório; portanto ele não pôde ser auditado.
- npm ci falhou por divergência entre package.json e package-lock.json. O bundle foi produzido com instalação local sem atualizar o lockfile.

Pontos fortes

Os controles abaixo foram verificados no código e contam como evidência positiva de cobertura.

Controle	Evidência
RLS é o mecanismo central e está ativado nas 15 tabelas de domínio.	As migrations habilitam RLS em profiles, teams, team_members, team_invites, repertorios, songs, repertorio_songs, cultos, culto_songs, culto_lineup, member_availability, notifications, messages, song_tracks e song_loop_points.
Entidades principais aplicam o papel correto no servidor.	songs/repertorios/cultos usam is_team_member para leitura e can_edit_team para escrita; times, membros e convites usam has_team_role(..., 'admin').
O bucket audio está isolado pelo primeiro segmento do caminho.	As quatro policies em 20260224005059...sql:10-24 usam storage.foldername(name)[1] e is_team_member/can_edit_team.
Operações por dono têm boas verificações em várias tabelas.	Disponibilidade valida team_member_id próprio; notificações de leitura/alteração/remoção validam profile_id; mensagens validam team_id e sender_profile_id; song_tracks deriva o time da música.
Handlers sensíveis possuem verificações úteis onde indicadas.	accept-invite valida JWT, token, estado e e-mail; public-playlist exige token UUID e is_public; separate-stems valida JWT e membership (embora o papel esteja fraco).
Worker interno falha fechado sem WORKER_TOKEN.	demucs-worker/app.py:31 usa os.environ['WORKER_TOKEN']; /jobs e downloads chamam _check com hmac.compare_digest. Somente /health é público por desenho.
Nenhum segredo privado hardcoded foi encontrado.	Árvore e dois commits foram examinados. .env contém apenas project ref/URL e JWT role=anon publicável; o bundle contém somente essa chave/URL esperadas. service_role, LOVABLE_API_KEY, WORKER_TOKEN e chaves privadas não aparecem no bundle nem no histórico.
Conteúdo textual é escapado pelo React.	Letras, cifras, chat, notas e notificações são interpolados como texto. Não há parser Markdown/HTML; o único dangerouslySetInnerHTML está em um componente de CSS de gráfico sem chamadores e sem dados de usuário.

Pontos fracos

- F-01 [Crítica] Tomada de conta por associação forçada a time e redefinição global de senha
- F-02 [Alta] Policies de Storage não isolam capas e escritas de avatar por usuário/time
- F-03 [Alta] Cinco funções pagas de IA estão públicas e sem rate limit
- F-04 [Alta] XSS armazenado por media_url aberto como javascript: em window.open
- F-05 [Alta] RLS valida apenas um lado de relações e permite vínculos entre tenants
- F-06 [Alta] Viewer pode iniciar separação de stems e alterar dados via service_role
- F-07 [Média] Callback autentica com Bearer undefined quando o segredo não existe
- F-08 [Média] RLS de notificações aceita destinatário/remetente arbitrários e team_id nulo
- F-09 [Média] Policy de status permite ao membro alterar todos os campos da escala

Cobertura sistemática de handlers e RLS

Todos os handlers backend

Tipo	Handler	Controle observado	Resultado
Edge	accept-invite	JWT manual; token + e-mail + accepted=false	Correto no escopo
Edge	admin-update-user	JWT + admin em time compartilhado	F-01: escopo global inseguro
Edge	create-invite	JWT + admin do team_id	F-01: membership sem aceite
Edge	import-cifra	Sem autenticação; IA paga	F-03
Edge	import-moises	Sem autenticação; IA paga	F-03
Edge	public-playlist	Público por token + is_public	Correto isoladamente; cadeia F-05
Edge	separate-stems	JWT + membership	F-06: papel insuficiente
Edge	stems-callback	Bearer compartilhado	F-07: fail-open se ausente
Edge	suggest-culto-songs	Sem autenticação; IA paga	F-03
Edge	sync-lyrics	Sem autenticação; IA paga	F-03
Edge	transpose	Sem autenticação; IA paga	F-03
FastAPI	GET /health	Público intencional; sem dados sensíveis	Correto no escopo
FastAPI	POST /jobs	Bearer WORKER_TOKEN + validação de URL	Correto no modelo service-to-service
FastAPI	GET /jobs/{jid}	Bearer WORKER_TOKEN; jid UUID aleatório	Correto no modelo service-to-service
FastAPI	GET /jobs/{jid}/stems/{stem}	Bearer WORKER_TOKEN + stem existente	Correto no modelo service-to-service



Tabelas e buckets — continuação da cobertura

Objeto	Mecanismo	Resultado
profiles	auth.uid() e co-membership	Leitura/alteração própria corretas; co-membro para perfil.
teams / team_members / team_invites	membership + papel admin	CRUD de tenant protegido; cadeia global em F-01.
repertorios / songs / cultos	team_id + is_team_member/can_edit_team	Políticas centrais corretas.
repertorio_songs / culto_songs	time derivado só do pai	F-05: lado song_id não validado.
culto_lineup	time derivado do culto; status por membro	F-05 e F-09.
member_availability	team_member próprio / time	SELECT e escritas por dono corretas.
notifications	profile dono; INSERT por editor	Leitura própria correta; INSERT vulnerável F-08.
messages	team_id + sender_profile_id	SELECT/INSERT/DELETE corretamente limitados.
song_tracks	time derivado de song_id	Leitura por membro; escrita por editor correta.
song_loop_points	profile dono / loop público no time	F-05: criação própria não valida time da música.
storage.audio	primeira pasta = team_id	Correto para SELECT/INSERT/UPDATE/DELETE.
storage.covers	somente authenticated	F-02: sem isolamento.
storage.avatars	leitura pública; escrita authenticated	F-02: escrita sem dono.



Tabela consolidada de achados

Severidade	Categoria	Arquivo:linha	Descrição
Crítica	Permissão definida no navegador	supabase/migrations/20260223230533_2b017539-a6b8-4380-832f-ed808dca9370.sql:111-127, 143-145	F-01 — Qualquer autenticado cria time; trigger o adiciona como admin.
Crítica	Permissão definida no navegador	supabase/functions/create-invite/index.ts:80-99, 132-155	F-01 — Conta existente é localizada, alterada e adicionada sem aceite.
Crítica	Permissão definida no navegador	supabase/migrations/20260224031925_6ef15eef-1a23-4908-9b41-515b22769b6c.sql; src/pages/TeamSettings.tsx:34-43; 50	F-01 — Após a associação, o atacante pode ler o user_id global da vítima como colega de time.
Crítica	Permissão definida no navegador	supabase/functions/admin-update-user/index.ts:78-113, 122-127	F-01 — Um time compartilhado habilita alteração da identidade global.
Alta	Banco sem tranca	supabase/migrations/20260223233111_7652ce77-5105-4938-ac16-00c2fa07302b.sql:19-40	F-02 — Todas as operações de covers dependem apenas de authenticated.
Alta	Banco sem tranca	supabase/migrations/20260224031925_6ef15eef-1a23-4908-9b41-515b22769b6c.sql:45-62	F-02 — Escritas de avatars não validam a pasta do usuário.
Alta	Banco sem tranca	src/components/CoverUpload.tsx; src/pages/TeamSettings.tsx:34-37; 111-118	F-02 — O frontend confirma namespaces teamId/songId e userId que as policies não usam.
Alta	Permissão definida no navegador	supabase/config.toml:6-16, 27-28	F-03 — JWT desativado para as cinco funções de IA.
Alta	Permissão definida no navegador	supabase/functions/import-cifra/index.ts; transpose/index.ts:9-13, 55-61; 9-13, 28-35	F-03 — O corpo é aceito antes de usar a chave; não há getClaims/getUser.
Alta	Permissão definida no navegador	supabase/functions/import-moises/index.ts; sync-lyrics/index.ts; suggest-culto-songs/index.ts:220-224, 283-292; 137-150, 187-190; 8-21, 53-56	F-03 — Os outros três handlers seguem o mesmo fluxo sem autenticação.
Alta	Inputs sem tratamento (XSS)	src/pages/SongEditor.tsx:153-171, 370-372	F-04 — Entrada de URL é persistida sem validação de esquema/host.
Alta	Inputs sem tratamento (XSS)	src/pages/Presentation.tsx:535-537, 1476-1487	F-04 — URL não-YouTube chega diretamente a window.open.
Alta	Inputs sem tratamento (XSS)	src/integrations/supabase/client.ts:11-16	F-04 — O impacto inclui a sessão persistida no localStorage.
Alta	IDOR	supabase/migrations/20260223230533_2b017539-a6b8-4380-832f-ed808dca9370.sql:239-253	F-05 — repertorio_songs deriva permissão só de repertorio_id; song_id não é comparado.
Alta	IDOR	supabase/functions/public-playlist/index.ts:25-28, 48-74	F-05 — service_role atravessa a relação inconsistente e assina a mídia.
Alta	IDOR	supabase/migrations/20260224031353_4e860432-1560-4d5d-8263-7291e2470a59.sql:41-51	F-05 — culto_songs valida o culto, mas não o time da música.
Alta	IDOR	supabase/migrations/20260224031925_6ef15eef-1a23-4908-9b41-515b22769b6c.sql:17-31	F-05 — culto_lineup valida culto_id, mas não o time de team_member_id.
Alta	IDOR	supabase/migrations/20260301201228_4702a9c9-71c3-48c7-87ed-6be03e776da9.sql:33-42	F-05 — Loops validam profile_id próprio, mas não membership no song_id.
Alta	Permissão definida no navegador	src/pages/SongEditor.tsx:97-103, 183, 457-466	F-06 — A UI usa canEdit/readOnly para esconder o botão.

Severidade	Categoria	Arquivo:linha	Descrição
Alta	Permissão definida no navegador	supabase/functions/separate-stems/index.ts:45-66, 79-104	F-06 — O backend testa membership, não role/can_edit_team.
Alta	Permissão definida no navegador	supabase/functions/stems-callback/index.ts:71-72, 93-106, 127-133	F-06 — O fluxo privilegiado remove/insere tracks e finaliza a música.
Média	Chaves expostas / configuração de segredo	supabase/config.toml:33-34	F-07 — O gateway não valida JWT para o callback.
Média	Chaves expostas / configuração de segredo	supabase/functions/stems-callback/index.ts:35-56	F-07 — A asserção ! desaparece em runtime e o valor previsível autentica.
Média	Banco sem tranca	src/components/AppLayout.tsx; src/components/SendNotificationModal.tsx:90-98; 40-49	F-08 — O gate canEdit está no navegador; o insert inclui IDs controláveis.
Média	Banco sem tranca	supabase/migrations/20260224230010_c9c85c73-e b68-48dc-892c-59f097844d12.sql:54-60	F-08 — team_id IS NULL ignora a verificação; recipient/sender não aparecem.
Média	Permissão definida no navegador	src/pages/Home.tsx:97-101	F-09 — A intenção da UI é alterar apenas status.
Média	Permissão definida no navegador	supabase/migrations/20260224230010_c9c85c73-e b68-48dc-892c-59f097844d12.sql:8-18	F-09 — A policy nomeada 'status' não restringe coluna nem culto/time.

Achados detalhados

CRÍTICA

Permissão definida no navegador

F-01 — Tomada de conta por associação forçada a time e redefinição global de senha

Um usuário autenticado pode criar um time, associar diretamente uma conta existente pelo e-mail sem aceite e então redefinir a senha global dessa conta como administrador do time recém-criado.

Por que é explorável

A policy permite que qualquer autenticado crie um time e o trigger o torna admin. create-invite encontra o usuário global pelo e-mail, altera seu perfil e insere a membership imediatamente. admin-update-user considera suficiente compartilhar qualquer time no qual o chamador seja admin e usa auth.admin.updateUserById. Como a identidade Supabase é global, o atacante assume também os outros times da vítima.

Condições: Requer uma conta autenticada, um time controlado pelo atacante e o e-mail de uma conta existente que seja retornada por listUsers(). Não exige que a vítima aceite convite nem que pertença previamente ao time do atacante.

Impacto

Comprometimento integral da conta, acesso aos demais times da vítima, alteração de dados e possibilidade de persistência. create-invite também pode alterar o nome global do perfil.

Evidências

supabase/migrations/20260223230533_2b017539-a6b8-4380-832f-ed808dca9370.sql:111-127, 143-145 — Qualquer autenticado cria time; trigger o adiciona como admin.

```
118 INSERT INTO public.team_members (team_id, profile_id, role)
119 VALUES (NEW.id, NEW.owner_id, 'admin');
...
143 CREATE POLICY "Authenticated users can create teams"
145 WITH CHECK (owner_id = (... auth.uid() ...));
```

supabase/functions/create-invite/index.ts:80-99, 132-155 — Conta existente é localizada, alterada e adicionada sem aceite.

```
81 const { data: existingUsers } = await admin.auth.admin.listUsers();
82 const existingUser = existingUsers?.users?.find(...);
88 if (existingUser) {
89   userId = existingUser.id;
...
133 if (full_name) {
134   await admin.from("profiles").update({ full_name }).eq("id", profile.id);
135 }
...
153 const { error: memberError } = await admin
154   .from("team_members")
155   .insert({ team_id, profile_id: profile.id, role });
```

supabase/migrations/20260224031925_6ef15eef-1a23-4908-9b41-515b22769b6c.sql; src/pages/TeamSettings.tsx:34-43; 50 — Após a associação, o atacante pode ler o user_id global da vítima como colega de time.

```
migration:34 CREATE POLICY "Team members can view teammate profiles"
migration:36 USING (EXISTS (... tm1.team_id = tm2.team_id ...));
```

```
TeamSettings.tsx:50 .select("id, role, profile_id, instruments,
                           profiles(id, full_name, user_id, avatar_url)")
```

supabase/functions/admin-update-user/index.ts:78-113, 122-127 — Um time compartilhado habilita alteração da identidade global.

```
78 // Check caller is admin in at least one shared team
94 const { data: targetMembership } = await admin
95   .from("team_members")
96   .in("team_id", callerTeamIds)
...
109 if (password) {
111   const { error: updateError } = await admin.auth.admin.updateUserById(
112     user_id, { password }
113   );
}
```

Correção recomendada

Transformar create-invite em convite pendente e exigir aceite pela conta destinatária; remover a redefinição administrativa de senha de identidades globais e usar o fluxo de recuperação enviado ao e-mail verificado. Não alterar perfil de usuário existente durante convite. Invalidar sessões e auditar usos anteriores dessas funções.

ALTA

Banco sem tranca

F-02 — Políticas de Storage não isolam capas e escritas de avatar por usuário/time

O bucket privado covers concede SELECT/INSERT/UPDATE/DELETE sobre qualquer objeto a todo usuário autenticado. Em avatars, qualquer autenticado pode sobrescrever ou excluir o arquivo de qualquer usuário.

Por que é explorável

Os caminhos já carregam teamId ou userId, mas as políticas ignoram essas pastas e testam somente auth.role(). Um atacante autenticado pode listar o bucket covers, gerar acesso aos objetos de outros times e sobrescrevê-los/excluí-los. Em avatars, a leitura pública é intencional, mas a escrita não está ligada ao auth.uid().

Condições: Requer sessão autenticada. Para alteração direcionada basta conhecer ou listar o caminho; a policy de SELECT de covers permite descoberta. Avatares são públicos e seus caminhos usam o user_id.

Impacto

Vazamento de capas privadas entre times, adulteração e exclusão de mídia, defacement de avatares e quebra do isolamento multi-tenant.

Evidências

supabase/migrations/20260223233111_7652ce77-5105-4938-ac16-00c2fa07302b.sql:19-40 — Todas as operações de covers dependem apenas de authenticated.

```
20 INSERT INTO storage.buckets (...) VALUES ('covers', 'covers', false);
...
28 CREATE POLICY "Authenticated users can view covers"
30 USING (bucket_id = 'covers' AND auth.role() = 'authenticated');
...
33 CREATE POLICY "Authenticated users can update covers"
35 USING (bucket_id = 'covers' AND auth.role() = 'authenticated');
38 CREATE POLICY "Authenticated users can delete covers"
40 USING (bucket_id = 'covers' AND auth.role() = 'authenticated');
```

supabase/migrations/20260224031925_6ef15eef-1a23-4908-9b41-515b22769b6c.sql:45-62 — Escritas de avatars não validam a pasta do usuário.

```
52 CREATE POLICY "Authenticated users can upload avatars"
54 WITH CHECK (bucket_id = 'avatars' AND auth.role() = 'authenticated');
56 CREATE POLICY "Users can update their own avatar"
58 USING (bucket_id = 'avatars' AND auth.role() = 'authenticated');
60 CREATE POLICY "Users can delete their own avatar"
62 USING (bucket_id = 'avatars' AND auth.role() = 'authenticated');
```

src/components/CoverUpload.tsx; src/pages/TeamSettings.tsx:34-37; 111-118 — O frontend confirma namespaces teamId/songId e userId que as políticas não usam.

```
CoverUpload.tsx:34 const path = `${teamId}/${songId}.${ext}`;
CoverUpload.tsx:37 ...from("covers").upload(path, file, { upsert: true });

TeamSettings.tsx:113 const path = `${profileUserId}/avatar.${ext}`;
TeamSettings.tsx:114 ...from("avatars").upload(path, file, { upsert: true });
```

Correção recomendada

Aplicar a mesma estratégia do bucket audio: extrair a primeira pasta e validar membership/can_edit_team. Em avatars, permitir escrita apenas na pasta auth.uid(); para edição por admin, usar função server-side que valide o time. Incluir WITH CHECK explícito em UPDATE.

ALTA

Permissão definida no navegador

F-03 — Cinco funções pagas de IA estão públicas e sem rate limit

import-cifra, import-moises, transpose, sync-lyrics e suggest-culto-songs desativam a verificação JWT e não fazem autenticação manual antes de consumir LOVABLE_API_KEY.

Por que é explorável

Os botões ficam dentro do app autenticado e, em vários fluxos, somente para editores, mas os endpoints podem ser chamados diretamente. Um terceiro usa a chave server-side como proxy de IA, enviando payloads repetidos/grandes até esgotar créditos ou indisponibilizar a funcionalidade.

Condições: As funções precisam estar implantadas com LOVABLE_API_KEY configurada. sync-lyrics não tem chamada no frontend atual, mas seu handler e configuração pública estão versionados.

Impacto

Esgotamento de créditos, aumento de custo, negação de serviço e abuso do provedor de IA.

Evidências

supabase/config.toml:6-16, 27-28 — JWT desativado para as cinco funções de IA.

```
6 [functions.import-cifra]
7   verify_jwt = false
9 [functions.transpose]
10  verify_jwt = false
12 [functions.sync-lyrics]
13  verify_jwt = false
15 [functions.import-moises]
16  verify_jwt = false
27 [functions.suggest-culto-songs]
28  verify_jwt = false
```

supabase/functions/import-cifra/index.ts; transpose/index.ts:9-13, 55-61; 9-13, 28-35 — O corpo é aceito antes de usar a chave; não há getClaims/getUser.

```
import-cifra:13 const { url } = await req.json();
import-cifra:55 const LOVABLE_API_KEY = Deno.env.get("LOVABLE_API_KEY");
import-cifra:58 const aiRes = await fetch("https://ai.gateway...", {

transpose:13 const { cifra_text, from_key, to_key } = await req.json();
transpose:31 const response = await fetch("https://ai.gateway...", {
```

supabase/functions/import-moises/index.ts; sync-lyrics/index.ts; suggest-culto-songs/index.ts:220-224, 283-292; 137-150, 187-190; 8-21, 53-56 — Os outros três handlers seguem o mesmo fluxo sem autenticação.

```
import-moises:224 const body = await req.json();
import-moises:289 const aiRes = await fetch("https://ai.gateway...", {
sync-lyrics:141 const { segments, duration_seconds } = await req.json();
sync-lyrics:187 const response = await fetch("https://ai.gateway...", {
suggest:12 const { liturgy, songCount, songs } = await req.json();
suggest:53 const response = await fetch("https://ai.gateway...", {
```

Correção recomendada

Ativar verify_jwt, validar a sessão no handler, exigir team_id/object_id e can_edit_team no servidor, impor limite de tamanho, quota/rate limit por usuário e time, e registrar consumo.

ALTA

Inputs sem tratamento (XSS)

F-04 — XSS armazenado por media_url aberto como javascript: em window.open

Um editor pode salvar media_url com esquema javascript:. Na apresentação, um membro que clicar em tocar mídia executa o código com acesso a window.opener.

Por que é explorável

media_url é salvo sem allowlist de protocolo. Se não for reconhecido como YouTube e a música não tiver áudio, Presentation chama window.open(mediaUrl, '_blank') sem noopener. O padrão foi reproduzido em Google Chrome headless: o javascript: alterou o DOM do opener, comprovando execução e acesso ao contexto da aplicação. A sessão Supabase persiste em localStorage.

Condições: Requer editor/admin malicioso, música sem audio_path, URL não reconhecida como YouTube e clique da vítima no botão de mídia.

Impacto

Execução arbitrária no navegador, roubo do JWT persistido, ações como a vítima e possível acesso a outros times da conta comprometida.

Evidências

src/pages/SongEditor.tsx:153-171, 370-372 — Entrada de URL é persistida sem validação de esquema/host.

```
163 media_url: mediaUrl.trim() || null,
...
370 <label>Link de mídia</label>
372 <GlassInput value={mediaUrl} onChange={(e) => setMediaUrl(e.target.value)} ... />
```

src/pages/Presentation.tsx:535-537, 1476-1487 — URL não-YouTube chega diretamente a window.open.

```
535 const mediaUrl = currentSong?.media_url || "";
536 const youtubeId = mediaUrl ? getYoutubeId(mediaUrl) : null;
537 const hasMedia = !!mediaUrl;
...
1483 if (youtubeId) {
1484   setShowMediaPlayer((v) => !v);
1485 } else {
1486   window.open(mediaUrl, "_blank");
1487 }
```

src/integrations/supabase/client.ts:11-16 — O impacto inclui a sessão persistida no localStorage.

```
11 export const supabase = createClient(..., {
12   auth: {
13     storage: localStorage,
14     persistSession: true,
15     autoRefreshToken: true,
```

Correção recomendada

Aceitar apenas URLs https válidas e, idealmente, provedores explicitamente permitidos; rejeitar javascript:, data:, file: e esquemas desconhecidos no cliente e no servidor. Abrir links com noopener,noreferrer e adicionar CSP como defesa em profundidade.

ALTA

IDOR

F-05 — RLS valida apenas um lado de relações e permite vínculos entre tenants

Policies de repertorio_songs, culto_songs, culto_lineup e song_loop_points autorizam o objeto pai/dono, mas não verificam se o ID relacionado pertence ao mesmo time.

Por que é explorável

Um editor pode associar a seu repertório/culto um song_id de outro tenant; pode também associar membro estrangeiro à escala. O caso de repertório vira vazamento: public-playlist usa service_role, lê o song relacionado sem RLS e assina capa/áudio. Loops próprios também aceitam qualquer song_id.

Condições: Para o vazamento principal, o atacante precisa ser editor de um time, conhecer um UUID de música de outro tenant e publicar o próprio repertório. UUIDs reduzem descoberta casual, mas não são controle de autorização. Os demais vetores causam integridade cruzada com IDs conhecidos.

Impacto

Leitura pública de título, artista, cifra/letra e URLs assinadas de mídia privada; poluição de relações entre tenants e notificações/escala para membros de outro time.

Evidências

supabase/migrations/20260223230533_2b017539-a6b8-4380-832f-ed808dca9370.sql:239-253 — repertorio_songs deriva permissão só de repertorio_id; song_id não é comparado.

```
243 CREATE POLICY "Editors can add repertorio songs"
245 WITH CHECK (public.can_edit_team(auth.uid(),
  (SELECT team_id FROM public.repertorios WHERE id = repertorio_id)));
247 CREATE POLICY "Editors can update repertorio songs"
249 USING (public.can_edit_team(auth.uid(),
  (SELECT team_id FROM public.repertorios WHERE id = repertorio_id)));
```

supabase/functions/public-playlist/index.ts:25-28, 48-74 — service_role atravessa a relação inconsistente e assina a mídia.

```
25 const supabase = createClient(
27   Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
28 );
...
49 const { data: repSongs } = await supabase
50   .from("repertorio_songs")
51   .select("...", songs(id, title, ..., cover_path, audio_path, ...))
...
64 await supabase.storage.from("covers").createSignedUrl(...)
```

```

72 await supabase.storage.from("audio").createSignedUrl(...)
supabase/migrations/20260224031353_4e860432-1560-4d5d-8263-7291e2470a59.sql:41-51 — culto_songs valida o culto, mas não o time da música.
44 CREATE POLICY "Editors can add culto songs" ...
45 WITH CHECK (can_edit_team(auth.uid(),
  (SELECT team_id FROM cultos WHERE id = culto_songs.culto_id)));
supabase/migrations/20260224031925_6ef15eef-1a23-4908-9b41-515b22769b6c.sql:17-31 — culto_lineup valida culto_id, mas não o time de team_member_id.
21 CREATE POLICY "Editors can insert culto lineup"
23 WITH CHECK (can_edit_team(auth.uid(),
  (SELECT cultos.team_id FROM cultos WHERE cultos.id = culto_lineup.culto_id)));
supabase/migrations/20260301201228_4702a9c9-71c3-48c7-87ed-6be03e776da9.sql:33-42 — Loops validam profile_id próprio, mas não membership no song_id.
33 CREATE POLICY "Users can create own loops"
36 WITH CHECK (profile_id = get_my_profile_id());
39 CREATE POLICY "Users can update own loops"
42 USING (profile_id = get_my_profile_id());

```

Correção recomendada

Em cada policy, validar os dois lados da FK e igualdade de team_id, com USING e WITH CHECK. Adicionar constraints/trigger de consistência quando possível. public-playlist deve revalidar team_id da música e selecionar apenas relações consistentes antes de assinar arquivos.

ALTA

Permissão definida no navegador

F-06 — Viewer pode iniciar separação de stems e alterar dados via service_role

O frontend oferece separação somente a admin/editor, mas separate-stems aceita qualquer membro do time e não limita repetição.

Por que é explorável

Viewers leem song_id e audio_path pelas policies de SELECT e podem chamar a função diretamente. O handler confere apenas a existência de membership, dispara o worker intensivo e usa service_role para marcar a música. O callback substitui tracks automáticas.

Condições: Requer conta viewer no time e worker/secrets configurados. Não depende da UI.

Impacto

Consumo repetido de CPU/tempo, negação de serviço, alteração de stems_status e substituição das faixas automáticas de uma música sem privilégio de edição.

Evidências

src/pages/SongEditor.tsx:97-103, 183, 457-466 — A UI usa canEdit/readOnly para esconder o botão.

```

101 const { data, error } = await supabase.functions.invoke("separate-stems", {
102   body: { song_id: id, audio_path: audioPath },
103 });
...
183 const readOnly = !canEdit;
458 {audioPath && !readOnly && (
460   <Button ... onClick={handleSeparateStems}>

```

supabase/functions/separate-stems/index.ts:45-66, 79-104 — O backend testa membership, não role/can_edit_team.

```

45 // Load the song and confirm the caller belongs to its team
60 const { data: membership } = await admin
61   .from("team_members")
63   .eq("team_id", song.team_id)
64   .eq("profile_id", callerProfile.id)
66 if (!membership) return json({ error: "Forbidden" }, 403);
...
80 const workerRes = await fetch(`${workerUrl}/jobs`, ...);
101 await admin.from("songs").update({ stems_status: "processing", ... })

```

supabase/functions/stems-callback/index.ts:71-72, 93-106, 127-133 — O fluxo privilegiado remove/insere tracks e finaliza a música.

```

72 await admin.from("song_tracks").delete().eq("song_id", songId)...;
93 const { error: upErr } = await admin.storage.from("audio").upload(...);
101 const { error: insErr } = await admin.from("song_tracks").insert(...);
127 await admin.from("songs").update({ stems_status: "done", ... })

```

Correção recomendada

Exigir role admin/editor no handler, comparar audio_path exatamente com song.audio_path, aplicar idempotência por música/job e rate limit por usuário/time.

MÉDIA

Chaves expostas / configuração de segredo

F-07 — Callback autentica com Bearer undefined quando o segredo não existe

stems-callback usa non-null assertion em STEMS_CALLBACK_TOKEN, que não valida em runtime. Se a variável estiver ausente, o segredo esperado vira a string pública Bearer undefined.

Por que é explorável

A função é implantada com verify_jwt=false. Um atacante envia Authorization: Bearer undefined, passa safeEqual e fornece meta.song_id arbitrário. O ramo status != done atualiza a tabela songs com service_role sem depender do worker.

Condições: Explorável somente em deploy no qual STEMS_CALLBACK_TOKEN esteja ausente (ou vazio com header equivalente aceito). separate-stems falha quando o segredo falta, mas o callback continua fail-open.

Impacto

Alteração cross-tenant de status/erro de músicas por ID conhecido, ignorando RLS.

Evidências

supabase/config.toml:33-34 — O gateway não valida JWT para o callback.

```
33 [functions.stems-callback]
34 verify_jwt = false
```

supabase/functions/stems-callback/index.ts:35-56 — A asserção ! desaparece em runtime e o valor previsível autentica.

```
36 const serviceKey = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY");
39 const callbackToken = Deno.env.get("STEMS_CALLBACK_TOKEN");
42 const auth = req.headers.get("Authorization") ?? "";
43 if (!safeEqual(auth, `Bearer ${callbackToken}`)) return json(..., 401);
...
45 const { job_id, status, stems, meta } = await req.json();
54 .from("songs").update({ stems_status: "error", ... }).eq("id", songId);
```

Correção recomendada

Validar presença, comprimento e entropia do segredo antes de comparar; se inválido, retornar 503 sem criar o client service_role. Preferir assinatura HMAC do corpo com timestamp e vincular job_id, song_id e team_id ao registro criado pelo servidor.

MÉDIA

Banco sem tranca

F-08 — RLS de notificações aceita destinatário/remetente arbitrários e team_id nulo

A UI reserva avisos a admin/editor, mas a policy de INSERT aprova qualquer linha com team_id NULL e não liga profile_id/sender_profile_id ao time nem ao chamador.

Por que é explorável

Um usuário autenticado pode chamar PostgREST diretamente, criar notificações para profile_id conhecido, fingir ser outro remetente e ignorar can_edit_team escolhendo team_id nulo.

Condições: Requer sessão autenticada e UUID do perfil alvo para ataque direcionado.

Impacto

Spam persistente, personificação e engenharia social entre tenants; crescimento indevido da tabela.

Evidências

src/components/AppLayout.tsx; src/components/SendNotificationModal.tsx:90-98; 40-49 — O gate canEdit está no navegador; o insert inclui IDs controláveis.

```
AppLayout.tsx:90 {canEdit && (
```

```
AppLayout.tsx:92    onClick={() => setNotifModalOpen(true)}
```

```
SendNotificationModal.tsx:40 const notifications = members.map((m) => ({
41   profile_id: m.profile_id,
42   team_id: currentTeam.id,
46   sender_profile_id: profileId,
49   ...from("notifications").insert(notifications as any);
```

supabase/migrations/20260224230010_c9c85c73-eb68-48dc-892c-59f097844d12.sql:54-60 — team_id IS NULL ignora a verificação; recipient/sender não aparecem.

```
55 CREATE POLICY "Team editors can create notifications"
56 ON public.notifications
57 FOR INSERT
58 WITH CHECK (
59   team_id IS NULL OR can_edit_team(auth.uid(), team_id)
60 );
```

Correção recomendada

Restringir a policy a authenticated, exigir team_id não nulo, can_edit_team, destinatário membro do mesmo time e sender_profile_id igual a get_my_profile_id(). Preferir RPC/Edge Function que derive remetente e destinatários no servidor.

MÉDIA

Permissão definida no navegador

F-09 — Policy de status permite ao membro alterar todos os campos da escala

O frontend atualiza somente status, porém a policy concede UPDATE da linha inteira quando team_member_id pertence ao chamador.

Por que é explorável

RLS limita quais linhas podem ser atualizadas, não quais colunas. Um viewer escalado pode enviar diretamente instrument, culto_id e outros campos. A condição só preserva a propriedade do team_member_id; não valida o culto nem restringe a mudança ao status.

Condições: Requer membro com ao menos uma linha de culto_lineup e os IDs desejados.

Impacto

Adulteração de escala e instrumentos; possível movimentação da própria escala para culto de outro tenant.

Evidências

src/pages/Home.tsx:97-101 — A intenção da UI é alterar apenas status.

```
97 const updateLineupStatus = async (lineupId, status) => {
98   const { error } = await supabase
99     .from("culto_lineup")
100     .update({ status } as any)
101     .eq("id", lineupId);
```

supabase/migrations/20260224230010_c9c85c73-eb68-48dc-892c-59f097844d12.sql:8-18 — A policy nomeada 'status' não restringe coluna nem culto/time.

```
9 CREATE POLICY "Members can update own lineup status"
10 ON public.culto_lineup
11 FOR UPDATE
12 USING (
13   team_member_id IN (
14     SELECT tm.id FROM team_members tm
16     WHERE p.user_id = auth.uid()
17   )
18 );
```

Correção recomendada

Revogar UPDATE direto para membros e expor RPC set_lineup_status(lineup_id, status), validando auth.uid(), enum permitido e mantendo todos os demais campos imutáveis. Alternativamente, usar privilégio de coluna e trigger defensivo, além de WITH CHECK de consistência de time.

Recomendações priorizadas

Prioridade	Ação	Resultado esperado
P1	Bloquear a cadeia de tomada de conta (F-01)	Remover reset global por admin e exigir aceite real do convite. Invalidar sessões e revisar logs.
P1	Fechar Storage e relações cross-tenant (F-02/F-05)	Publicar migration com policies de ambos os lados, WITH CHECK e testes com dois tenants.
P1	Eliminar o XSS de media_url (F-04)	Allowlist HTTPS/hosts no servidor e cliente; noopener/noreferrer e CSP defensiva.
P1	Autenticar e limitar funções de IA (F-03)	JWT, papel por time, quotas, limites de payload e rate limit antes de nova implantação.
P2	Alinhar stems ao papel editor (F-06)	can_edit_team server-side, caminho exato, idempotência e quota de jobs.
P2	Fazer callback falhar fechado (F-07)	Validação obrigatória do segredo, HMAC/timestamp e vínculo do job à música.
P2	Endurecer notificações e escala (F-08/F-09)	RPCs estreitas, remetente derivado da sessão e alteração apenas de status.
P3	Criar suíte de autorização	Testes automatizados por tabela/bucket/handler com admin, editor, viewer, anon e dois tenants.
P3	Corrigir drift de schema e build	Reconstituir migrations vazias/colunas ausentes e sincronizar package-lock.json para npm ci reproduzível.

Verificações técnicas executadas

- Build Vite de produção: concluído com sucesso (3464 módulos).
- Bundle: contém a URL e a chave Supabase role=anon esperadas; zero ocorrência de service_role, LOVABLE_API_KEY, WORKER_TOKEN, STEMS_CALLBACK_TOKEN ou padrões de chave privada.
- Histórico Git: dois commits alcançáveis examinados; nenhum segredo privado confirmado.
- Teste unitário existente: 1 arquivo / 1 teste aprovado.
- Lint: falhou com 170 erros e 25 avisos preexistentes, predominantemente no-explicit-any e dependências de hooks.
- Reprodução XSS: Chrome headless confirmou execução de javascript: aberto por window.open e acesso ao opener.

ISSUES PARA O GITHUB

Os blocos abaixo estão completos e prontos para copiar e colar. Achados relacionados foram agrupados: Storage em F-02, funções de IA em F-03 e relações multi-tenant em F-05.

--- ISSUE 1 ---

```
# [Segurança] Tomada de conta por associação forçada a time e redefinição global de senha

**Labels sugeridas:** `security`, `severidade:critica`

## Descrição do problema
```

Um usuário autenticado pode criar um time, associar diretamente uma conta existente pelo e-mail sem aceite e então redefinir a senha global dessa conta como administrador do time recém-criado.

A policy permite que qualquer autenticado crie um time e o trigger o torna admin. create-invite encontra o usuário global pelo e-mail, altera seu perfil e insere a membership imediatamente. admin-update-user considera suficiente compartilhar qualquer time no qual o chamador seja admin e usa auth.admin.updateUserById. Como a identidade Supabase é global, o atacante assume também os outros times da vítima.

Condições de explorabilidade: Requer uma conta autenticada, um time controlado pelo atacante e o e-mail de uma conta existente que seja retornada por listUsers(). Não exige que a vítima aceite convite nem que pertença previamente ao time do atacante.

```
## Evidência

- `supabase/migrations/20260223230533_2b017539-a6b8-4380-832f-ed808dca9370.sql:111-127, 143-145` – Qualquer autenticado cria time; trigger o adiciona como admin.
```

```
```text
118 INSERT INTO public.team_members (team_id, profile_id, role)
119 VALUES (NEW.id, NEW.owner_id, 'admin');
...
143 CREATE POLICY "Authenticated users can create teams"
144 WITH CHECK (owner_id = (... auth.uid() ...));
...

- `supabase/functions/create-invite/index.ts:80-99, 132-155` – Conta existente é localizada, alterada e adicionada sem aceite.

```text
81 const { data: existingUsers } = await admin.auth.admin.listUsers();
82 const existingUser = existingUsers?.users?.find(...);
83 if (existingUser) {
84   userId = existingUser.id;
...
133 if (full_name) {
134   await admin.from("profiles").update({ full_name }).eq("id", profile.id);
135 }
...
153 const { error: memberError } = await admin
154   .from("team_members")
155   .insert({ team_id, profile_id: profile.id, role });
...

- `supabase/migrations/20260224031925_6ef15eef-1a23-4908-9b41-515b22769b6c.sql; src/pages/TeamSettings.tsx:34-43; 50` – Após a associação, o atacante pode ler o user_id global da vítima como colega de time.
```

```
```text
migration:34 CREATE POLICY "Team members can view teammate profiles"
migration:36 USING (EXISTS (... tm1.team_id = tm2.team_id ...));

TeamSettings.tsx:50 .select("id, role, profile_id, instruments,
 profiles(id, full_name, user_id, avatar_url)")
...

- `supabase/functions/admin-update-user/index.ts:78-113, 122-127` – Um time compartilhado habilita alteração da identidade global.
```

```
```text
78 // Check caller is admin in at least one shared team
94 const { data: targetMembership } = await admin
95   .from("team_members")
96   .in("team_id", callerTeamIds)
...
109 if (password) {
110   const { error: updateError } = await admin.auth.admin.updateUserById(
111     user_id, { password }
112   )
...

```

```
113    });
114  }
115  ...
```

Impacto

Comprometimento integral da conta, acesso aos demais times da vítima, alteração de dados e possibilidade de persistência. create-invite também pode alterar o nome global do perfil.

Sugestão de correção

Transformar create-invite em convite pendente e exigir aceite pela conta destinatária; remover a redefinição administrativa de senha de identidades globais e usar o fluxo de recuperação enviado ao e-mail verificado. Não alterar perfil de usuário existente durante convite. Invalidar sessões e auditar usos anteriores dessas funções.

Critérios de aceite

- [] Um admin não consegue inserir diretamente uma conta existente no próprio time.
- [] A membership só nasce após aceite autenticado e e-mail compatível com o convite.
- [] Nenhum admin de time consegue trocar a senha ou o perfil global de outro usuário.
- [] O teste de regressão criar time -> forçar membership -> trocar senha termina em 403.
- [] Sessões da conta não são afetadas por operações administrativas de outro time.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Policies de Storage não isolam capas e escritas de avatar por usuário/time

****Labels sugeridas:**** `security`, `severidade:alta`

Descrição do problema

O bucket privado covers concede SELECT/INSERT/UPDATE/DELETE sobre qualquer objeto a todo usuário autenticado. Em avatars, qualquer autenticado pode sobrescrever ou excluir o arquivo de qualquer usuário.

Os caminhos já carregam teamId ou userId, mas as policies ignoram essas pastas e testam somente auth.role(). Um atacante autenticado pode listar o bucket covers, gerar acesso aos objetos de outros times e sobrescrevê-los/excluí-los. Em avatars, a leitura pública é intencional, mas a escrita não está ligada ao auth.uid().

****Condições de explorabilidade:**** Requer sessão autenticada. Para alteração direcionada basta conhecer ou listar o caminho; a policy de SELECT de covers permite descoberta. Avatares são públicos e seus caminhos usam o user_id.

Evidência

- `supabase/migrations/20260223233111\_7652ce77-5105-4938-ac16-00c2fa07302b.sql:19-40` — Todas as operações de covers dependem apenas de authenticated.

```text

```
20 INSERT INTO storage.buckets (...) VALUES ('covers', 'covers', false);
```

...

```
28 CREATE POLICY "Authenticated users can view covers"
```

```
30 USING (bucket_id = 'covers' AND auth.role() = 'authenticated');
```

...

```
33 CREATE POLICY "Authenticated users can update covers"
```

```
35 USING (bucket_id = 'covers' AND auth.role() = 'authenticated');
```

```
38 CREATE POLICY "Authenticated users can delete covers"
```

```
40 USING (bucket_id = 'covers' AND auth.role() = 'authenticated');
```

...

- `supabase/migrations/20260224031925\_6ef15eef-1a23-4908-9b41-515b22769b6c.sql:45-62` — Escritas de avatars não validam a pasta do usuário.

```text

```
52 CREATE POLICY "Authenticated users can upload avatars"
```

```
54 WITH CHECK (bucket_id = 'avatars' AND auth.role() = 'authenticated');
```

```
56 CREATE POLICY "Users can update their own avatar"
```

```
58 USING (bucket_id = 'avatars' AND auth.role() = 'authenticated');
```

```
60 CREATE POLICY "Users can delete their own avatar"
```

```
62 USING (bucket_id = 'avatars' AND auth.role() = 'authenticated');
```

...

- `src/components/CoverUpload.tsx; src/pages/TeamSettings.tsx:34-37; 111-118` — O frontend confirma namespaces teamId/songId e userId que as policies não usam.

```text

```
CoverUpload.tsx:34 const path = `${teamId}/${songId}.${ext}`;
```

```
CoverUpload.tsx:37 ...from("covers").upload(path, file, { upsert: true });
```

```
TeamSettings.tsx:113 const path = `${profileUserId}/avatar.${ext}`;
TeamSettings.tsx:114 ...from("avatars").upload(path, file, { upsert: true });
...
```

### ## Impacto

Vazamento de capas privadas entre times, adulteração e exclusão de mídia, defacement de avatares e quebra do isolamento multi-tenant.

### ## Sugestão de correção

Aplicar a mesma estratégia do bucket audio: extrair a primeira pasta e validar membership/can\_edit\_team. Em avatars, permitir escrita apenas na pasta auth.uid(); para edição por admin, usar função server-side que valide o time. Incluir WITH CHECK explícito em UPDATE.

### ## Critérios de aceite

- [ ] Membro do time A não lista, assina, altera nem exclui capas do time B.
- [ ] Viewer não envia/atualiza capa; admin/editor do time correto consegue.
- [ ] Usuário só grava e remove objetos na própria pasta de avatar.
- [ ] UPDATE não permite mover objeto para namespace de outro time/usuário.
- [ ] Testes automatizados cobrem SELECT, INSERT, UPDATE e DELETE nos três buckets.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

# [Segurança] Cinco funções pagas de IA estão públicas e sem rate limit

\*\*Labels sugeridas:\*\* `security`, `severidade:alta`

### ## Descrição do problema

import-cifra, import-moises, transpose, sync-lyrics e suggest-culto-songs desativam a verificação JWT e não fazem autenticação manual antes de consumir LOVABLE\_API\_KEY.

Os botões ficam dentro do app autenticado e, em vários fluxos, somente para editores, mas os endpoints podem ser chamados diretamente. Um terceiro usa a chave server-side como proxy de IA, enviando payloads repetidos/grandes até esgotar créditos ou indisponibilizar a funcionalidade.

\*\*Condições de explorabilidade:\*\* As funções precisam estar implantadas com LOVABLE\_API\_KEY configurada. sync-lyrics não tem chamada no frontend atual, mas seu handler e configuração pública estão versionados.

### ## Evidência

- `supabase/config.toml:6-16, 27-28` – JWT desativado para as cinco funções de IA.

```text

```
6 [functions.import-cifra]
7   verify_jwt = false
9 [functions.transpose]
10  verify_jwt = false
12 [functions.sync-lyrics]
13  verify_jwt = false
15 [functions.import-moises]
16  verify_jwt = false
27 [functions.suggest-culto-songs]
28  verify_jwt = false
...
```

- `supabase/functions/import-cifra/index.ts; transpose/index.ts:9-13, 55-61; 9-13, 28-35` – O corpo é aceito antes de usar a chave; não há getClaims/getUser.

```text

```
import-cifra:13 const { url } = await req.json();
import-cifra:55 const LOVABLE_API_KEY = Deno.env.get("LOVABLE_API_KEY");
import-cifra:58 const aiRes = await fetch("https://ai.gateway...", {
```

```
transpose:13 const { cifra_text, from_key, to_key } = await req.json();
transpose:31 const response = await fetch("https://ai.gateway...", {
...
```

- `supabase/functions/import-moises/index.ts; sync-lyrics/index.ts; suggest-culto-songs/index.ts:220-224, 283-292; 137-150, 187-190; 8-21, 53-56` – Os outros três handlers seguem o mesmo fluxo sem autenticação.

```text

```
import-moises:224 const body = await req.json();
import-moises:289 const aiRes = await fetch("https://ai.gateway...", {
sync-lyrics:141 const { segments, duration_seconds } = await req.json();
sync-lyrics:187 const response = await fetch("https://ai.gateway...", {
suggest:12 const { liturgy, songCount, songs } = await req.json();
```

```

suggest:53      const response = await fetch("https://ai.gateway...", {
  ...

  ## Impacto

  Esgotamento de créditos, aumento de custo, negação de serviço e abuso do provedor de IA.

  ## Sugestão de correção

  Ativar verify_jwt, validar a sessão no handler, exigir team_id/object_id e can_edit_team
no servidor, impor limite de tamanho, quota/rate limit por usuário e time, e registrar consumo.

  ## Critérios de aceite

  - [ ] Chamadas sem JWT válido retornam 401 em todas as cinco funções.
- [ ] Viewer recebe 403 nos recursos exclusivos de edição.
- [ ] team_id e objeto são validados server-side, sem confiar no payload de músicas.
- [ ] Há limite de tamanho e rate limit testável por usuário/time/IP.
- [ ] Testes cobrem chamadas diretas fora da UI.

  --- FIM ISSUE 3 ---

--- ISSUE 4 ---

# [Segurança] XSS armazenado por media_url aberto como javascript: em window.open

**Labels sugeridas:** `security`, `severidade:alta`

## Descrição do problema

Um editor pode salvar media_url com esquema javascript:. Na apresentação, um membro que
clicar em tocar mídia executa o código com acesso a window.opener.

media_url é salvo sem allowlist de protocolo. Se não for reconhecido como YouTube e a
música não tiver áudio, Presentation chama window.open(mediaUrl, '_blank') sem noopener. O padrão
foi reproduzido em Google Chrome headless: o javascript: alterou o DOM do opener, comprovando
execução e acesso ao contexto da aplicação. A sessão Supabase persiste em localStorage.

**Condições de explorabilidade:** Requer editor/admin malicioso, música sem audio_path,
URL não reconhecida como YouTube e clique da vítima no botão de mídia.

## Evidência

- `src/pages/SongEditor.tsx:153-171, 370-372` – Entrada de URL é persistida sem validação
de esquema/host.

```text
163 media_url: mediaUrl.trim() || null,
...
370 <label>Link de mídia</label>
372 <GlassInput value={mediaUrl} onChange={(e) => setMediaUrl(e.target.value)} ... />
```

- `src/pages/Presentation.tsx:535-537, 1476-1487` – URL não-YouTube chega diretamente a window.
open.

```text
...
535 const mediaUrl = currentSong?.media_url || "";
536 const youtubeId = mediaUrl ? getYoutubeId(mediaUrl) : null;
537 const hasMedia = !!mediaUrl;
...
1483 if (youtubeId) {
1484 setShowMediaPlayer((v) => !v);
1485 } else {
1486 window.open(mediaUrl, "_blank");
1487 }
```

- `src/integrations/supabase/client.ts:11-16` – O impacto inclui a sessão persistida no
localStorage.

```text
...
11 export const supabase = createClient(..., {
12 auth: {
13 storage: localStorage,
14 persistSession: true,
15 autoRefreshToken: true,
...

Impacto

Execução arbitrária no navegador, roubo do JWT persistido, ações como a vítima e possível
acesso a outros times da conta comprometida.

```

## ## Sugestão de correção

Aceitar apenas URLs https válidas e, idealmente, provedores explicitamente permitidos; rejeitar javascript:, data:, file: e esquemas desconhecidos no cliente e no servidor. Abrir links com noopener,noreferrer e adicionar CSP como defesa em profundidade.

## ## Critérios de aceite

- [ ] media\_url com javascript:, data: ou esquema desconhecido é rejeitado antes de persistir.
- [ ] Somente https e hosts permitidos podem ser abertos.
- [ ] Links externos usam noopener,noreferrer e não expõem window.opener.
- [ ] Teste E2E comprova que payload javascript: não executa e não lê localStorage.
- [ ] YouTube e provedores aprovados continuam funcionando.

--- FIM ISSUE 4 ---

## --- ISSUE 5 ---

# [Segurança] RLS valida apenas um lado de relações e permite vínculos entre tenants

\*\*Labels sugeridas:\*\* `security`, `severidade:alta`

## ## Descrição do problema

Policies de repertorio\_songs, culto\_songs, culto\_lineup e song\_loop\_points autorizam o objeto pai/dono, mas não verificam se o ID relacionado pertence ao mesmo time.

Um editor pode associar a seu repertório/culto um song\_id de outro tenant; pode também associar membro estrangeiro à escala. O caso de repertório vira vazamento: public-playlist usa service\_role, lê o song relacionado sem RLS e assina capa/áudio. Loops próprios também aceitam qualquer song\_id.

\*\*Condições de explorabilidade:\*\* Para o vazamento principal, o atacante precisa ser editor de um time, conhecer um UUID de música de outro tenant e publicar o próprio repertório. UUIDs reduzem descoberta casual, mas não são controle de autorização. Os demais vetores causam integridade cruzada com IDs conhecidos.

## ## Evidência

- `supabase/migrations/20260223230533\_2b017539-a6b8-4380-832f-ed808dca9370.sql:239-253` – repertorio\_songs deriva permissão só de repertorio\_id; song\_id não é comparado.

```
```text
```

```
243 CREATE POLICY "Editors can add repertorio songs"
245 WITH CHECK (public.can_edit_team(auth.uid(),
  (SELECT team_id FROM public.repertorios WHERE id = repertorio_id)));
247 CREATE POLICY "Editors can update repertorio songs"
249 USING (public.can_edit_team(auth.uid(),
  (SELECT team_id FROM public.repertorios WHERE id = repertorio_id)));
...`
```

- `supabase/functions/public-playlist/index.ts:25-28, 48-74` – service_role atravessa a relação inconsistente e assina a mídia.

```
```text
```

```
25 const supabase = createClient(
27 Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
28);
...
49 const { data: repSongs } = await supabase
50 .from("repertorio_songs")
51 .select("...", songs(id, title, ..., cover_path, audio_path, ...))
...
64 await supabase.storage.from("covers").createSignedUrl(...)
72 await supabase.storage.from("audio").createSignedUrl(...)
...`
```

- `supabase/migrations/20260224031353\_4e860432-1560-4d5d-8263-7291e2470a59.sql:41-51` – culto\_songs valida o culto, mas não o time da música.

```
```text
```

```
44 CREATE POLICY "Editors can add culto songs" ...
45 WITH CHECK (can_edit_team(auth.uid(),
  (SELECT team_id FROM cultos WHERE id = culto_songs.culto_id)));
...`
```

- `supabase/migrations/20260224031925\_6ef15eef-1a23-4908-9b41-515b22769b6c.sql:17-31` – culto_lineup valida culto_id, mas não o time de team_member_id.

```
```text
```

```
21 CREATE POLICY "Editors can insert culto lineup"
23 WITH CHECK (can_edit_team(auth.uid(),
 (SELECT cultos.team_id FROM cultos WHERE cultos.id = culto_lineup.culto_id)));
...`
```

```

...
- `supabase/migrations/20260301201228_4702a9c9-71c3-48c7-87ed-6be03e776da9.sql:33-42` — Loops
validam profile_id próprio, mas não membership no song_id.

```text
33 CREATE POLICY "Users can create own loops"
36 WITH CHECK (profile_id = get_my_profile_id());
39 CREATE POLICY "Users can update own loops"
42 USING (profile_id = get_my_profile_id());
```

Impacto

Leitura pública de título, artista, cifra/letra e URLs assinadas de mídia privada;
poluição de relações entre tenants e notificações/escala para membros de outro time.

Sugestão de correção

Em cada policy, validar os dois lados da FK e igualdade de team_id, com USING e WITH
CHECK. Adicionar constraints/trigger de consistência quando possível. public-playlist deve
revalidar team_id da música e selecionar apenas relações consistentes antes de assinar arquivos.

Critérios de aceite

- [] song_id de time B é rejeitado em repertório/culto de time A.
- [] team_member_id de time B é rejeitado em culto de time A.
- [] Loop só pode apontar para música de time do usuário.
- [] public-playlist nunca retorna música cujo team_id difira do repertório.
- [] Testes usam UUIDs conhecidos entre dois tenants e esperam zero linhas/403.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Viewer pode iniciar separação de stems e alterar dados via service_role

Labels sugeridas: `security`, `severidade:alta`

Descrição do problema

O frontend oferece separação somente a admin/editor, mas separate-stems aceita qualquer
membro do time e não limita repetição.

Viewers leem song_id e audio_path pelas policies de SELECT e podem chamar a função
diretamente. O handler confere apenas a existência de membership, dispara o worker intensivo e
usa service_role para marcar a música. O callback substitui tracks automáticas.

Condições de explorabilidade: Requer conta viewer no time e worker/secrets
configurados. Não depende da UI.

Evidência

- `src/pages/SongEditor.tsx:97-103, 183, 457-466` — A UI usa canEdit/readOnly para
esconder o botão.

```text
101 const { data, error } = await supabase.functions.invoke("separate-stems", {
102   body: { song_id: id, audio_path: audioPath },
103 });
...
183 const readOnly = !canEdit;
458 {audioPath && !readOnly && (
460   <Button ... onClick={handleSeparateStems}>
...
- `supabase/functions/separate-stems/index.ts:45-66, 79-104` — O backend testa membership, não
role/can_edit_team.

```text
45 // Load the song and confirm the caller belongs to its team
60 const { data: membership } = await admin
61 .from("team_members")
63 .eq("team_id", song.team_id)
64 .eq("profile_id", callerProfile.id)
66 if (!membership) return json({ error: "Forbidden" }, 403);
...
80 const workerRes = await fetch(`${workerUrl}/jobs`, ...);
101 await admin.from("songs").update({ stems_status: "processing", ... })
...
- `supabase/functions/stems-callback/index.ts:71-72, 93-106, 127-133` — O fluxo privilegiado
remove/insere tracks e finaliza a música.

```

```

```text
72 await admin.from("song_tracks").delete().eq("song_id", songId)...;
93 const { error: upErr } = await admin.storage.from("audio").upload(...);
101 const { error: insErr } = await admin.from("song_tracks").insert(...);
127 await admin.from("songs").update({ stems_status: "done", ... })
```

Impacto

Consumo repetido de CPU/tempo, negação de serviço, alteração de stems_status e substituição das faixas automáticas de uma música sem privilégio de edição.

Sugestão de correção

Exigir role admin/editor no handler, comparar audio_path exatamente com song.audio_path, aplicar idempotência por música/job e rate limit por usuário/time.

Critérios de aceite

- [] Viewer recebe 403 ao chamar separate-stems diretamente.
- [] Admin/editor do time correto consegue iniciar um job.
- [] audio_path diferente do persistido é rejeitado.
- [] Uma música em processing não aceita jobs paralelos não autorizados.
- [] Quota/rate limit impede enfileiramento abusivo.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

[Segurança] Callback autentica com Bearer undefined quando o segredo não existe

Labels sugeridas: `security`, `severidade:média`

Descrição do problema

stems-callback usa non-null assertion em STEMS_CALLBACK_TOKEN, que não valida em runtime. Se a variável estiver ausente, o segredo esperado vira a string pública Bearer undefined.

A função é implantada com verify_jwt=false. Um atacante envia Authorization: Bearer undefined, passa safeEqual e fornece meta.song_id arbitrário. O ramo status != done atualiza a tabela songs com service_role sem depender do worker.

Condições de explorabilidade: Explorável somente em deploy no qual STEMS_CALLBACK_TOKEN esteja ausente (ou vazio com header equivalente aceito). separate-stems falha quando o segredo falta, mas o callback continua fail-open.

Evidência

- `supabase/config.toml:33-34` – O gateway não valida JWT para o callback.

```text
33 [functions.stems-callback]
34 verify_jwt = false
```

- `supabase/functions/stems-callback/index.ts:35-56` – A asserção ! desaparece em runtime e o valor previsível autentica.

```text
36 const serviceKey = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;
39 const callbackToken = Deno.env.get("STEMS_CALLBACK_TOKEN")!;
42 const auth = req.headers.get("Authorization") ?? "";
43 if (!safeEqual(auth, `Bearer ${callbackToken}`)) return json(..., 401);
...
45 const { job_id, status, stems, meta } = await req.json();
54 .from("songs").update({ stems_status: "error", ... }).eq("id", songId);
```

Impacto

Alteração cross-tenant de status/erro de músicas por ID conhecido, ignorando RLS.

Sugestão de correção

Validar presença, comprimento e entropia do segredo antes de comparar; se inválido, retornar 503 sem criar o client service_role. Preferir assinatura HMAC do corpo com timestamp e vincular job_id, song_id e team_id ao registro criado pelo servidor.

Critérios de aceite

- [] Sem STEMS_CALLBACK_TOKEN, qualquer requisição retorna 503 e nenhum dado muda.
- [] Bearer undefined, vazio ou incorreto retorna 401 quando o serviço está configurado.

```

- [ ] Callback só aceita job\_id vinculado à música e ao time persistidos.
- [ ] Payload adulterado/repetido é rejeitado.
- [ ] Deploy valida o segredo antes de publicar a função.

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

# [Segurança] RLS de notificações aceita destinatário/remetente arbitrários e team\_id nulo

\*\*Labels sugeridas:\*\* `security`, `severidade:média`

## Descrição do problema

A UI reserva avisos a admin/editor, mas a policy de INSERT aprova qualquer linha com team\_id NULL e não liga profile\_id/sender\_profile\_id ao time nem ao chamador.

Um usuário autenticado pode chamar PostgREST diretamente, criar notificações para profile\_id conhecido, fingir ser outro remetente e ignorar can\_edit\_team escolhendo team\_id nulo.

\*\*Condições de explorabilidade:\*\* Requer sessão autenticada e UUID do perfil alvo para ataque direcionado.

## Evidência

- `src/components/AppLayout.tsx; src/components/SendNotificationModal.tsx:90-98; 40-49` — O gate canEdit está no navegador; o insert inclui IDs controláveis.

```text

```
AppLayout.tsx:90 {canEdit && (
AppLayout.tsx:92   onClick={() => setNotifModalOpen(true)}
```

```
SendNotificationModal.tsx:40 const notifications = members.map((m) => ({
41   profile_id: m.profile_id,
42   team_id: currentTeam.id,
46   sender_profile_id: profileId,
49   ...from("notifications").insert(notifications as any);
...`
```

- `supabase/migrations/20260224230010\_c9c85c73-eb68-48dc-892c-59f097844d12.sql:54-60` — team_id IS NULL ignora a verificação; recipient/sender não aparecem.

```text

```
55 CREATE POLICY "Team editors can create notifications"
56 ON public.notifications
57 FOR INSERT
58 WITH CHECK (
59 team_id IS NULL OR can_edit_team(auth.uid(), team_id)
60);
...`
```

## Impacto

Spam persistente, personificação e engenharia social entre tenants; crescimento indevido da tabela.

## Sugestão de correção

Restringir a policy a authenticated, exigir team\_id não nulo, can\_edit\_team, destinatário membro do mesmo time e sender\_profile\_id igual a get\_my\_profile\_id(). Preferir RPC/Edge Function que derive remetente e destinatários no servidor.

## Critérios de aceite

- [ ] INSERT com team\_id NULL é rejeitado.
- [ ] Viewer não envia avisos mesmo chamando PostgREST diretamente.
- [ ] Destinatário precisa pertencer ao team\_id informado.
- [ ] sender\_profile\_id é derivado da sessão e não pode ser forjado.
- [ ] Testes cobrem dois tenants e profile\_id conhecido.

--- FIM ISSUE 8 ---

--- ISSUE 9 ---

# [Segurança] Policy de status permite ao membro alterar todos os campos da escala

\*\*Labels sugeridas:\*\* `security`, `severidade:média`

## Descrição do problema

O frontend atualiza somente status, porém a policy concede UPDATE da linha inteira quando



team\_member\_id pertence ao chamador.

RLS limita quais linhas podem ser atualizadas, não quais colunas. Um viewer escalado pode enviar diretamente instrument, culto\_id e outros campos. A condição só preserva a propriedade do team\_member\_id; não valida o culto nem restringe a mudança ao status.

**\*\*Condições de explorabilidade:\*\*** Requer membro com ao menos uma linha de culto\_lineup e os IDs desejados.

#### ## Evidência

- `src/pages/Home.tsx:97-101` – A intenção da UI é alterar apenas status.

```
```text
```

```
97 const updateLineupStatus = async (lineupId, status) => {
  98   const { error } = await supabase
  99     .from("culto_lineup")
  100     .update({ status } as any)
  101     .eq("id", lineupId);
  ```
```

- `supabase/migrations/20260224230010\_c9c85c73-eb68-48dc-892c-59f097844d12.sql:8-18` – A policy nomeada 'status' não restringe coluna nem culto/time.

```
```text
```

```
9 CREATE POLICY "Members can update own lineup status"
10 ON public.culto_lineup
11 FOR UPDATE
12 USING (
13   team_member_id IN (
14     SELECT tm.id FROM team_members tm
15     WHERE p.user_id = auth.uid()
16   )
17 );
18 );
```
```

#### ## Impacto

Adulteração de escala e instrumentos; possível movimentação da própria escala para culto de outro tenant.

#### ## Sugestão de correção

Revogar UPDATE direto para membros e expor RPC set\_lineup\_status(lineup\_id, status), validando auth.uid(), enum permitido e mantendo todos os demais campos imutáveis. Alternativamente, usar privilégio de coluna e trigger defensivo, além de WITH CHECK de consistência de time.

#### ## Critérios de aceite

- [ ] Viewer consegue alterar pending para accepted/declined na própria linha.
- [ ] Viewer não altera instrument, culto\_id, team\_member\_id ou created\_at.
- [ ] Viewer não move escala para outro tenant.
- [ ] Admin/editor mantém o fluxo legítimo de gestão da escala.
- [ ] Teste direto via PostgREST valida cada coluna proibida.

--- FIM ISSUE 9 ---